

Penerapan Algoritma Dijkstra dan BFS pada Hospital Patient Routing

Nama Anggota:

Adi Satria Pangestu	5027231043
Muhammad Rafi Pramudya Putra	5027251024
Daffa Ulhaq Fadhlurrahman	5027251033
Muhammad Nadhif Pasya Ikhsan	5027251084
Michiko Artika Satriyo	5027251105

1. Deskripsi Masalah

Operasional rumah sakit berskala besar memiliki tantangan kompleksitas tinggi dalam mengatur pergerakan pasien karena melibatkan puluhan unit pelayanan fisik termasuk IGD, ruang triase, poli spesialis, laboratorium, apotek, hingga kasir yang letaknya saling berjauhan. Ketidakefisienan mobilisasi pasien ini menjadi permasalahan serius jika pasien kesulitan menentukan urutan ruangan yang harus dikunjungi sesuai prosedur medis yang harus mereka lalui serta rute terbaik untuk menuju lokasi berikutnya. Tanpa adanya sistem pemetaan denah yang terintegrasi memicu pembengkakan waktu tunggu dan waktu transit antar-fasilitas sehingga menurunkan efisiensi pelayanan rumah sakit.

Situasi ini menjadi lebih rumit ketika sistem dihadapkan pada manajemen antrian dinamis dan kasus gawat darurat. Sistem antrian seperti *First-Come, First-Served* (FCFS) tidak lagi relevan diterapkan dalam kondisi kesehatan pasien yang bersifat dinamis dan bervariasi sesuai tingkat keparahannya. Saat pasien dalam kondisi kritis masuk ke sistem, pasien tersebut harus segera diprioritaskan untuk mendapatkan penanganan utama. Hal ini menimbulkan konsekuensi tergesernya jadwal pelayanan pasien biasa, sehingga sistem harus mampu melakukan penjadwalan ulang secara otomatis oleh simulasi berbasis waktu (*event-driven*).

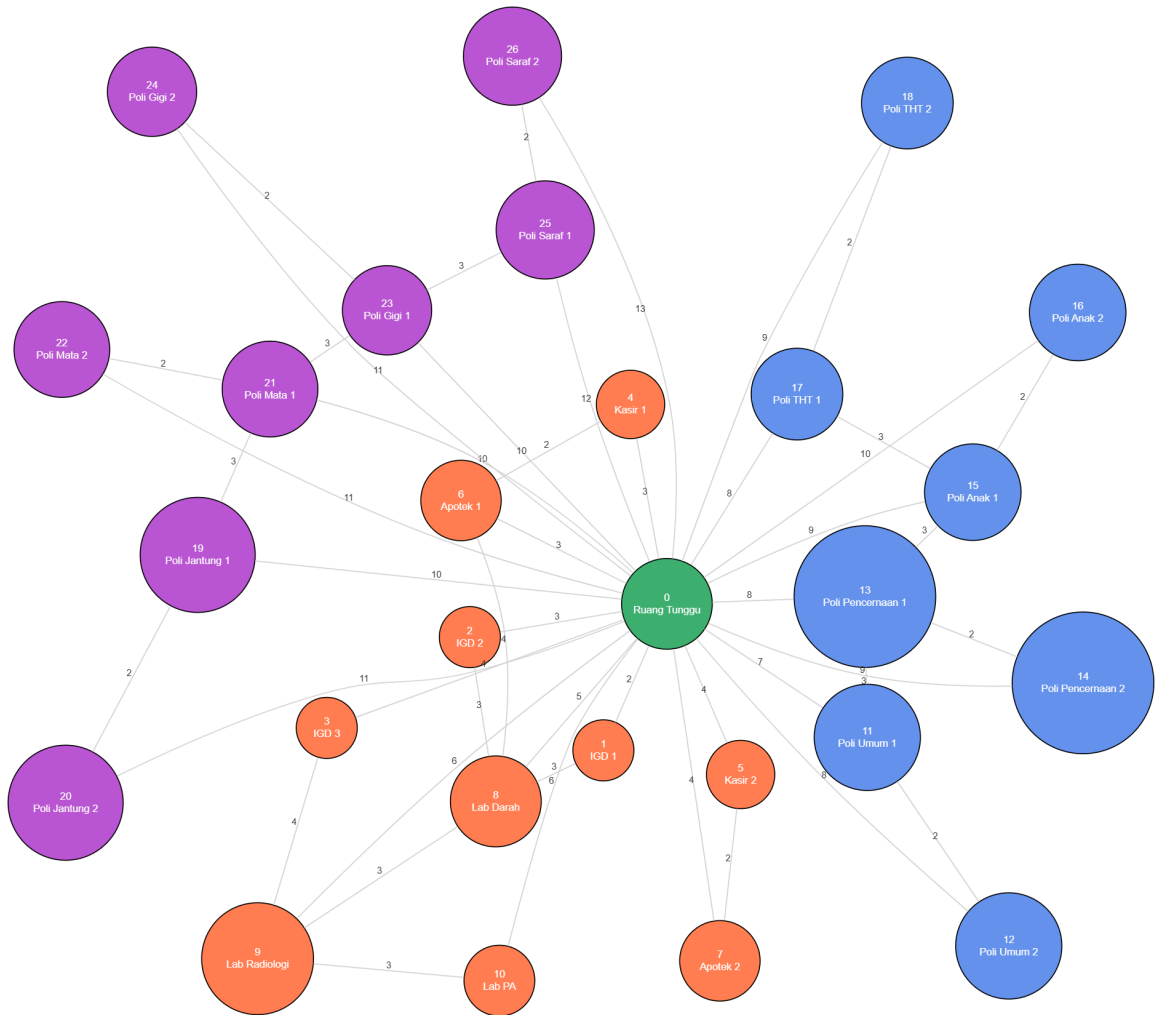
Untuk mengatasi sinkronisasi rute internal dan manajemen prioritas medis ini, diperlukan rancangan sistem yang memadukan struktur data Tree dan Graph. Struktur Graph dengan Adjacency List digunakan untuk memodelkan denah rumah sakit, dengan titik layanan sebagai node dan koneksi antar-ruangan (beserta bobot menitnya) sebagai edge. Algoritma Dijkstra digunakan untuk mencari rute dengan waktu minimum bagi pasien kritis, sementara BFS memetakan alur layanan pasien umum. Struktur Tree berupa Max-Heap diimplementasikan di setiap ruangan untuk mengelola antrian prioritas agar pasien kritis otomatis menempati posisi root (teratas).

2. Dataset:

Strukdat FP dataset

3. Struktur Graph

Graph yang digunakan dalam sistem ini adalah *undirected weighted graph*, dimana setiap node merepresentasikan satu ruangan di rumah sakit, setiap edge merepresentasikan jalur yang dapat dilalui pasien dari satu ruangan ke ruangan lainnya, dan setiap weight merepresentasikan waktu tempuh. Visualisasi dari graph yang digunakan dapat dilihat pada gambar berikut:



Graph direpresentasikan menggunakan Adjacency List melalui dua struktur utama di dalam kelas Graph.java:

- Map<Integer, Ruangan> nodes menyimpan seluruh data ruangan dengan key berupa ID integer. Setiap value adalah objek Ruangan yang memiliki atribut id, nama, kategori,

subKategori, rataRataLayanan, kapasitas, lantai, tersediaPada, dan antrian (MaxHeapAntrian). Contohnya seperti nodes.get(0) yang mengembalikan objek Ruang Tunggu Utama.

- Map<Integer, List<int[]>> adj menyimpan daftar tetangga tiap node. Key adalah ID node, value adalah array int[] berisi {nodeId_tetangga, bobot_waktu}. Contohnya adalah isi adj.get(0) adalah list yang berisi [1,2], [2,3], [3,4], [4,3], ... yang artinya node 0 terhubung ke node 1 dengan bobot 2, ke node 2 dengan bobot 3, dan seterusnya. Karena graph bersifat undirected, setiap pemanggilan tambahEdge(a, b, w) menambahkan entry di adj.get(a) sekaligus adj.get(b).

4. Struktur Tree

4.1 MaxHeapAntrian

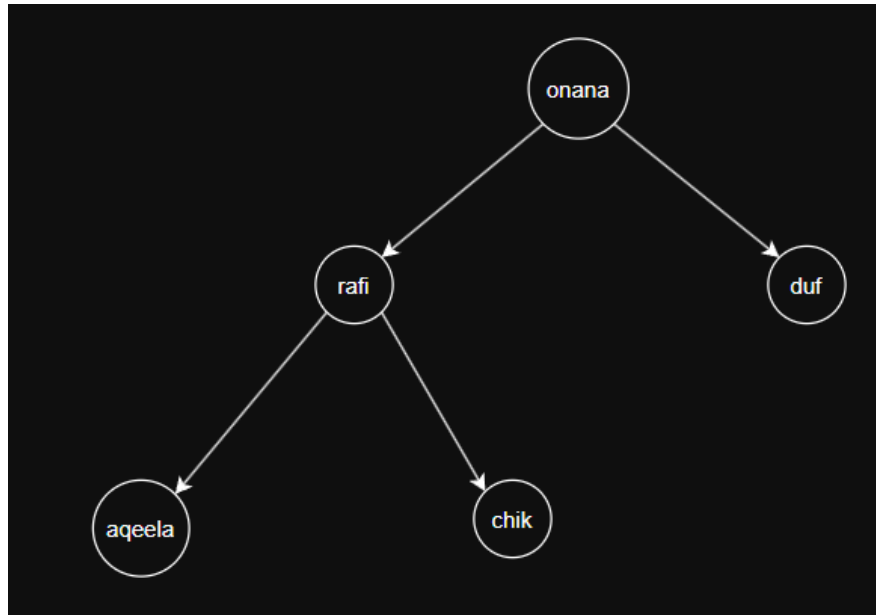
Tree ini digunakan untuk menentukan prioritas pasien. Penentuan prioritas ini dapat dilihat dari fungsi lebihUtama():

```
private boolean lebihUtama(Pasien a, Pasien b) {
    if (a.prioritas != b.prioritas)
        return a.prioritas < b.prioritas;
    return a.waktuMasuk < b.waktuMasuk;
}
```

Artinya, pasien DARURAT (nilai prioritas = 1) selalu berada di atas pasien biasa (prioritas = 2). Jika dua pasien memiliki prioritas yang sama, maka pasien dengan waktuMasuk lebih kecil (lebih dulu datang) yang berada di posisi lebih atas.

Contoh implementasi dari tree berikut dapat dilihat melalui data dibawah ini:

Node	Nama	Status	Prioritas	Waktu Masuk
0 (root)	onana	Darurat	1	45
1	rafi	Normal	2	22
2	duf	Normal	2	40
3	aqeela	Normal	2	68
4	chik	Normal	2	77



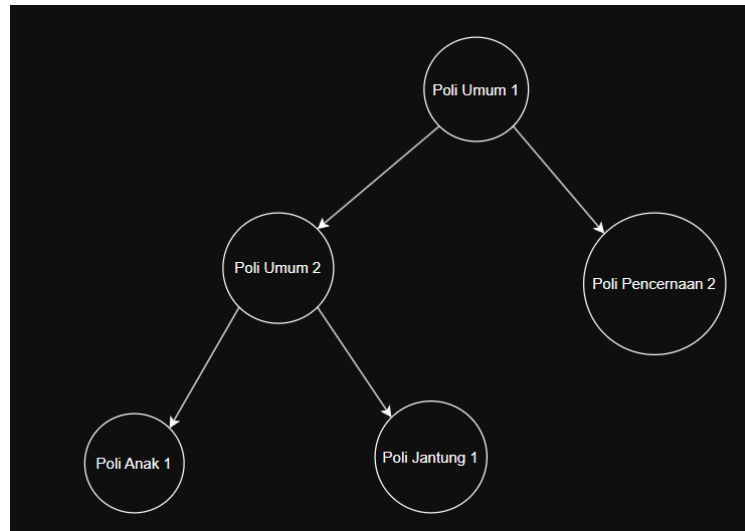
4.2 MinHeapDijkstra

Tree digunakan sebagai untuk membantu algoritma Dijkstra mencari rute terdekat. Fungsi utama dari tree ini adalah sebagai berikut:

- insert(nodeId, jarak) : Menambahkan node baru ke dalam heap, dilanjutkan dengan proses *heapify-up* untuk menata ulang posisi berdasarkan jarak terkecil.
- extractMin(): Mengambil sekaligus menghapus node dengan jarak terkecil dari posisi *root*, dilanjutkan dengan proses *heapify-down* untuk menaikkan elemen terkecil berikutnya.

Contoh penerapannya adalah saat pasien Rafi mencari rute menuju APOTEK dari posisi Node 11 (Poli Umum 1) dapat dilihat melalui data di bawah ini:

Posisi di Heap	ID Node	Nama Ruangan	Jarak dari Node 11
0	6	Apotek 1	10
1	7	Apotek 2	11
2	0	Ruang tunggu utama	7
3	4	Kasir 1	9
4	8	Lab darah	12



Munculnya beberapa ruangan lain (seperti Kasir atau Lab Darah) di dalam data antrean terjadi karena sifat algoritma Dijkstra yang mengeksplorasi seluruh cabang jalan yang terhubung langsung dengan Ruang Tunggu (Node 0). Semua cabang tersebut didaftarkan ke dalam heap tanpa pilih kasih, lalu diurutkan secara otomatis berdasarkan total akumulasi jaraknya. Ruangan dengan bobot jalur terkecil akan otomatis terapung ke indeks teratas heap, sedangkan rute alternatif lainnya tetap mengantre di bawahnya (visualisasi hanya menggunakan 5 node untuk mempermudah visualisasi, untuk selengkapnya bisa dilihat pada visualisasi Graph).

5. Algoritma

Secara garis Besar kita memiliki beberapa mekanisme algoritma untuk menangani case dari **Hospital Patient Routing**, Berikut penjabarannya.

A. Algoritma BFS

Jadi disini kami menggunakan algoritma BFS untuk mencari Kandidat node yang dituju. Semisal ada pasien baru mendaftar maka dia akan berada di node start (Ruang Utama) terlebih dahulu, lalu user akan memilih kasus apa yang ingin dituju, contohnya disini pasien masuk dengan keadaan non darurat, Keluhan sakit tertentu dan sakit jantung. Lalu program akan membuat list sop urutan dari penyakit tersebut.

Rencana alur layanan untuk rafi:

1. POLI (JANTUNG)
2. LAB
3. APOTEK
4. KASIR

Terlihat digambar pasien dari start (Ruang Tunggu Utama) ingin lanjut ke poli Jantung, Program Bfs akan mencari node node yang memiliki kategori Poli Jantung.

```
[t=4] rafi ? mencari POLI (JANTUNG) dari posisi node #0 (Ruang Tunggu Utama)
Kandidat ruangan:
- Poli Jantung 1 | tempuh=10 | antrian=0 org | tunggu=0 | total=10
- Poli Jantung 2 | tempuh=11 | antrian=0 org | tunggu=0 | total=11
[Dipilih] Poli Jantung 1 | rute: Ruang Tunggu Utama ? Poli Jantung 1 | mulai t=4
```

B. Algoritma Dijkstra

Terlihat dari gambar Bfs berhasil menemukan 2 node dengan kategori Poli Jantung, ada Poli Jantung 1 dan Poli Jantung 2. Lalu untuk menentukan rute tercepat menuju Poli tersebut menggunakan metode dijkstra. Output dari algoritma tersebut adalah waktu tempuh tiap node. Lalu program akan mendeklarasikan waktu total tiap node dengan rumus $total = waktuTempuh + EstimasiTunggu$

C. Greedy

Setelah mendapatkan total waktu tempuh tiap node ruangan , program akan melakukan perbandingan total waktu secara greedy untuk memperoleh node yang akan dituju kedepannya. Sehingga diperoleh seperti Gambar

6. Design Decision Log

No	Keputusan	Alternatif	Alasan Memilih	Risiko/Kelemahan
1	Menggunakan Breadth-First Search	Depth-First Search	Sangat efektif untuk mencari graf secara mendatar	Harus mengalokasikan memori tambahan untuk antrian
2	Menggunakan Algoritma Dijkstra	For Loop Tetangga	Posisi ruangan pasien berubah ubah sehingga rute terpendek berikutnya perlu dihitung ulang	Kompleksitas waktu komputasi meningkat
3	Menggunakan Max-Heap	PriorityQueue	Memudahkan pembuatan fungsi snapshotUrutan() untuk menampilkan kondisi antrian secara urut	
4	Menggunakan Min-Heap Kustom	Array List	Mengoptimalkan performa algoritma	Memerlukan struktur kelas

			Dijkstra dengan mempercepat pencarian node	terpisah dan manajemen indeks berbasis yang lebih rumit untuk diatur.
5	Menggunakan Antrian di Setiap Node	Berbagi Satu Antrian Global untuk Semua Ruangan	Memudahkan membuat sistem antrian tiap node	Membutuhkan alokasi memori tersendiri di dalam setiap objek ruangan
6	Menggunakan Antrian Urutan Kejadian	<i>Real-Time Loop</i>	Memudahkan dalam mengatur pengecekan antrian tiap node	Perhitungan sinkronisasi tidak terjadi secara realtime jadi refresh tiap program merefresh.
7	Menggunakan Adjacency List	Adjacency Matrix	Denah rumah sakit bertipe <i>sparse graph</i> sehingga <i>adjacency list</i> jauh lebih menghemat ruang memori.	Proses pengecekan hubungan langsung antar dua node menjadi sedikit lebih lambat
8	Menggunakan <i>Greedy</i> untuk mencari Rute Terbaik	Evaluasi sisa seluruh alur SOP	Mempercepat pengambilan keputusan rute karena	-
9	Menggunakan Array untuk Representasi Edge	Membuat Kelas Khusus Objek Edge	Mengurangi beban pembuatan objek baru	Struktur data menjadi kurang deskriptif secara pemrograman berorientasi OOP
10	Menggunakan Tracker Kunjungan Hash set	ArrayList	Menjamin efisiensi waktu pengecekan status kunjungan node dalam waktu konstan $O(1)$.	Membutuhkan konsumsi memori yang relatif lebih besar

7. Tracing

☰ Tracing Heap, Graph, dan Edge pada Hospital Patient Routing

- **Tambah Pasien**

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
```

```
PS C:\Users\ASUS\Downloads\StrukdatHospitalPatientRouting-main> & 'C:\Program Files\Eclipse Adoptium\jdk-17.0.18-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' -cp 'C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\8dfb089829b7d12f5b365be58e90af42\redhat.java\jdt_ws\StrukdatHospitalPatientRouting-main_811026f7\bin\' 'src.Main'
```

```
----- SISTEM ROUTING PASIEN RS -----
```

```
1. Tambahkan Pasien
2. Lihat antrian semua ruangan
3. Tampilkan struktur graph
0. Keluar
Pilih: 1

Nama pasien: Danang
Apakah pasien dalam kondisi darurat?
1. Ya
2. Tidak
Pilih: 2
Apa keperluan pasien
1. Cek kesehatan umum
2. Ada sakit tertentu
Pilih: 2
Sakit apa?
1. Pencernaan (maag, sakit perut)
2. Jantung (sesak, sakit dada)
3. Anak (demam pada anak)
4. Mata
5. Gigi
6. THI (telinga, hidung, tenggorokan)
7. Saraf (pusing, migrain)
Pilih: 2

Rencana alur layanan untuk Danang:
1. POLI (JANTUNG)
2. LAB
3. APOTEK
4. KASIR

Pasien Danang (ID=1) didaftarkan pada t=64

[t=64] Danang ? mencari POLI (JANTUNG) dari posisi node 0 (Ruangan Utama)
Kandidat ruangan:
- Poli Jantung 1 | tempuh=10 | antrian=0 org | tunggu=0 | total=10
```

```

----- SISTEM ROUTING PASIEN RS -----
1. Tambahkan Pasien
2. Lihat antrian semua ruangan
3. Tampilkan struktur graph
0. Keluar
Pilih: 1

Nama pasien: Luky
Apakah pasien dalam kondisi darurat?
1. Ya
2. Tidak
Pilih: 1

Setelah di IGD, pasien dirujuk ke:
1. Jantung
2. Saraf
3. THT
4. Anak
5. Mata
Pilih: 1

Rencana alur layanan untuk Luky:
1. IGD
2. POLI (JANTUNG)
3. APOTEK
4. KASIR

Pasien Luky (ID=2) didaftarkan pada t=101

[t=101] Luky ? mencari IGD dari posisi node 0 (Ruang Tunggu Utama)
Kandidat ruangan:
- IGD 1 | tempuh=2 | antrian=0 org | tunggu=0 | total=2
- IGD 2 | tempuh=3 | antrian=0 org | tunggu=0 | total=3
- IGD 3 | tempuh=4 | antrian=0 org | tunggu=0 | total=4
[Dipilih] IGD 1 | rute: Ruang Tunggu Utama ? IGD 1 | mulai t=101 | selesai t=161

```

```

Nama pasien: andi
Apakah pasien dalam kondisi darurat?
1. Ya
2. Tidak
Pilih: 2
Apa keperluan pasien
1. Cek kesehatan umum
2. Ada sakit tertentu
Pilih: 1

Rencana alur layanan untuk andi:
1. POLI (UMUM)
2. APOTEK
3. KASIR

Pasien andi (ID=3) didaftarkan pada t=301

[t=301] andi ? mencari POLI (UMUM) dari posisi node 0 (Ruang Tunggu Utama)
Kandidat ruangan:
- Poli Umum 1 | tempuh=7 | antrian=0 org | tunggu=0 | total=7
- Poli Umum 2 | tempuh=8 | antrian=0 org | tunggu=0 | total=8
[Dipilih] Poli Umum 1 | rute: Ruang Tunggu Utama ? Poli Umum 1 | mulai t=301 | selesai t=331

[EVENT t=301] Luky selesai di Poli Jantung 1.

[t=301] Luky ? mencari APOTEK dari posisi node 19 (Poli Jantung 1)
Kandidat ruangan:
- Apotek 1 | tempuh=13 | antrian=0 org | tunggu=0 | total=13
- Apotek 2 | tempuh=14 | antrian=0 org | tunggu=0 | total=14
[Dipilih] Apotek 1 | rute: Poli Jantung 1 ? Ruang Tunggu Utama ? Apotek 1 | mulai t=301 | selesai t=326

[EVENT t=301] Danang selesai di Lab Darah.

[t=301] Danang ? mencari APOTEK dari posisi node 8 (Lab Darah)
Kandidat ruangan:
- Apotek 1 | tempuh=4 | antrian=1 org | tunggu=21 | total=25
- Apotek 2 | tempuh=5 | antrian=0 org | tunggu=0 | total=5

```


● Lihat Antrian Semua Ruangan

```
----- SISTEM ROUTING PASIEN RS -----
1. Tambahkan Pasien
2. Lihat antrian semua ruangan
3. Tampilkan struktur graph
0. Keluar
Pilih: 2

===== ANTRIAN SEMUA RUANGAN (t=524) =====

Kasir 1:
[MENUNGGU] 1 pasien:
- Luky (DARURAT) | mulai t=334 | selesai t=354

Kasir 2:
[MENUNGGU] 1 pasien:
- Danang (Normal) | mulai t=334 | selesai t=354

Apotek 1:
[MENUNGGU] 1 pasien:
- andi (Normal) | mulai t=334 | selesai t=359

[EVENT t=524] Danang selesai di Kasir 2.

[SELESAI t=524] Danang telah menyelesaikan seluruh layanan!

[EVENT t=524] Luky selesai di Kasir 1.

[SELESAI t=524] Luky telah menyelesaikan seluruh layanan!

[EVENT t=524] andi selesai di Apotek 1.

[t=524] andi ? mencari KASIR dari posisi node 6 (Apotek 1)
Kandidat ruangan:
- Kasir 1 | tempuh=2 | antrian=0 org | tunggu=0 | total=2
- Kasir 2 | tempuh=7 | antrian=0 org | tunggu=0 | total=7
[Dipilih] Kasir 1 | rute: Apotek 1 ? Kasir 1 | mulai t=524 | selesai t=544
```

● Tampilkan Struktur Graph

```
----- SISTEM ROUTING PASIEN RS -----
1. Tambahkan Pasien
2. Lihat antrian semua ruangan
3. Tampilkan struktur graph
0. Keluar
Pilih: 3

Level 0:
- Ruang Tunggu Utama [RUANG_TUNGGU, lantai 1]

Level 1:
- IGD 1 [IGD, lantai 1]
- IGD 2 [IGD, lantai 1]
- IGD 3 [IGD, lantai 1]
- Kasir 1 [KASIR, lantai 1]
- Kasir 2 [KASIR, lantai 1]
- Apotek 1 [APOTEK, lantai 1]
- Apotek 2 [APOTEK, lantai 1]
- Lab Darah [LAB, lantai 1]
- Lab Radiologi [LAB, lantai 1]
- Lab PA [LAB, lantai 1]
- Poli Umum 1 [POLI/UMUM, lantai 2]
- Poli Umum 2 [POLI/UMUM, lantai 2]
- Poli Pencernaan 1 [POLI/PENCERNAAN, lantai 2]
- Poli Pencernaan 2 [POLI/PENCERNAAN, lantai 2]
- Poli Anak 1 [POLI/ANAK, lantai 2]
- Poli Anak 2 [POLI/ANAK, lantai 2]
- Poli THT 1 [POLI/THT, lantai 2]
- Poli THT 2 [POLI/THT, lantai 2]
- Poli Jantung 1 [POLI/JANTUNG, lantai 3]
- Poli Jantung 2 [POLI/JANTUNG, lantai 3]
- Poli Mata 1 [POLI/MATA, lantai 3]
- Poli Mata 2 [POLI/MATA, lantai 3]
- Poli Gigi 1 [POLI/GIGI, lantai 3]
- Poli Gigi 2 [POLI/GIGI, lantai 3]
- Poli Saraf 1 [POLI/SARAF, lantai 3]
- Poli Saraf 2 [POLI/SARAF, lantai 3]
```

```
- Poli Anak 1 [POLI/ANAK, lantai 2]
- Poli Anak 2 [POLI/ANAK, lantai 2]
- Poli THT 1 [POLI/THT, lantai 2]
- Poli THT 2 [POLI/THT, lantai 2]
- Poli Jantung 1 [POLI/JANTUNG, lantai 3]
- Poli Jantung 2 [POLI/JANTUNG, lantai 3]
- Poli Mata 1 [POLI/MATA, lantai 3]
- Poli Mata 2 [POLI/MATA, lantai 3]
- Poli Gigi 1 [POLI/GIGI, lantai 3]
- Poli Gigi 2 [POLI/GIGI, lantai 3]
- Poli Saraf 1 [POLI/SARAF, lantai 3]
- Poli Saraf 2 [POLI/SARAF, lantai 3]
```

[EVENT t=334] Luky selesai di Apotek 1.

[t=334] Luky ? mencari KASIR dari posisi node 6 (Apotek 1)

Kandidat ruangan:

```
- Kasir 1 | tempuh=2 | antrian=0 org | tunggu=0 | total=2
- Kasir 2 | tempuh=7 | antrian=0 org | tunggu=0 | total=7
```

[Dipilih] Kasir 1 | rute: Apotek 1 ? Kasir 1 | mulai t=334 | selesai t=354

[EVENT t=334] Danang selesai di Apotek 2.

[t=334] Danang ? mencari KASIR dari posisi node 7 (Apotek 2)

Kandidat ruangan:

```
- Kasir 2 | tempuh=2 | antrian=0 org | tunggu=0 | total=2
- Kasir 1 | tempuh=7 | antrian=1 org | tunggu=13 | total=20
```

[Dipilih] Kasir 2 | rute: Apotek 2 ? Kasir 2 | mulai t=334 | selesai t=354

[EVENT t=334] andi selesai di Poli Umum 1.

[t=334] andi ? mencari APOTEK dari posisi node 11 (Poli Umum 1)

Kandidat ruangan:

```
- Apotek 1 | tempuh=10 | antrian=0 org | tunggu=0 | total=10
- Apotek 2 | tempuh=11 | antrian=0 org | tunggu=0 | total=11
```

[Dipilih] Apotek 1 | rute: Poli Umum 1 ? Ruang Tunggu Utama ? Apotek 1 | mulai t=334 | selesai t=359

9. Analisis Kompleksitas

Komponen Fitur	Algoritma / Struktur Data	Kompleksitas Waktu (Big-O)	Alasan
Pencarian Poliklinik	BFS (<i>Adjacency List</i>)	$O(V + E)$	Mengunjungi semua vertex dan edge
Pencarian Rute Tercepat	Dijkstra + Kustom Min-Heap	$O((V + E) \log V)$	Menggunakan min-heap untuk ekstraksi jarak minimum
Penataan Jadwal Layanan	recalculateSchedule+ Snapshot	$O(N \log N)$	Melakukan sorting snapshot urutan antrian

10. What If Analysis

Bagaimana jika pasien darurat masuk secara terus-menerus?

- **Mekanisme:** MaxHeapAntrian selalu memprioritaskan pasien darurat dan fungsi recalculateSchedule akan terus menyusun ulang antrian.
- **Dampak Utama:** Terjadi **starvation**. Pasien dengan prioritas normal akan terus tergeser ke belakang dan secara teoritis berisiko tidak pernah mendapatkan pelayanan.

Bagaimana jika terdapat dua ruangan dengan total bobot yang persis sama?

- **Mekanisme:** Evaluasi kandidat ruangan di fungsi prosesNextTahap menggunakan operator kurang dari yang ketat ($<$).
- **Dampak Utama:** Tidak ada **load balancing**. Sistem bersifat bias dan akan selalu memilih ruangan pertama yang ditemukan dalam iterasi BFS, mengabaikan kandidat lain yang memiliki efisiensi identik.

Bagaimana jika kapasitas ruangan terlampaui oleh antrian pasien?

- **Mekanisme:** Atribut kapasitas pada objek Ruangan sudah diinisialisasi, tetapi tidak ada validasi penolakan di dalam fungsi `prosesNextTahap`.
- **Dampak Utama:** Terjadi **unbounded queue** (antrian tak terbatas). Sistem gagal menyimulasikan batasan fisik dunia nyata karena tidak adanya mekanisme *fallback* (pengalihan otomatis) jika antrian poli sudah penuh.

11. Kesimpulan

Secara keseluruhan, sistem penjadwalan dan *routing* rumah sakit yang dibangun menggunakan kombinasi struktur data Graf, algoritma BFS, Dijkstra, dan arsitektur *Event-Driven* telah berhasil menyimulasikan penyelesaian masalah antrian dengan efektif. Sistem ini unggul dalam memadukan aspek spasial dan temporal, memungkinkan pencarian poliklinik secara terarah, optimalisasi rute tercepat dengan *Min-Heap* kustom, serta manajemen prioritas pasien darurat secara *real-time* melalui *Max-Heap*. Meskipun dasar arsitekturnya sudah solid, evaluasi *What-If Analysis* menunjukkan perlunya beberapa pengembangan lebih lanjut untuk menyempurnakan batasan sistem saat ini. Pengembangan di masa depan tersebut meliputi penambahan mekanisme *Aging* untuk mencegah terjadinya *starvation* pada pasien prioritas normal, penerapan *load balancing* agar beban antrian merata pada ruangan dengan bobot layanan yang identik, serta implementasi validasi kapasitas ruangan untuk mencegah penumpukan pasien tanpa batas (*unbounded queue*) yang tidak sesuai dengan kondisi fisik nyata rumah sakit.